



Hoja de referencia rápida de programación ESP32

Guía de referencia rápida para instrucciones y funciones del curso de Informática Industrial

Por [Juan M. Gandarias](#) | jmgandarias@uma.es

0 Configuración inicial y preparación

Estructura básica del programa

```
const int LED_PIN = 2;

void setup() {
  Serial.begin(115200);
  delay(1000);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("ESP32 inicializado");
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```



Comunicación serie (depuración)

Función	Descripción
<code>Serial.begin(baudrate)</code>	Inicializa el puerto serie (se recomienda 115200)
<code>Serial.print()</code>	Muestra texto sin salto de línea
<code>Serial.println()</code>	Muestra texto con salto de línea
<code>Serial.printf()</code>	Muestra texto con formato
<code>Serial.available()</code>	Bytes disponibles en el buffer
<code>Serial.read()</code>	Lee un byte
<code>Serial.readString()</code>	Lee una línea completa

1 E/S digitales, analógicas y PWM (LEDC)



GPIO y lectura analógica

Función	Descripción
<code>pinMode(pin, mode)</code>	Configura el pin como INPUT , OUTPUT o INPUT_PULLUP (activa la resistencia interna pull-up)
<code>digitalWrite(pin, val)</code>	Establece el estado de salida digital (HIGH o LOW)
<code>digitalRead(pin)</code>	Lee el estado lógico del pin → HIGH (1) o LOW (0)
<code>analogRead(pin)</code>	Conversión A/D → Devuelve un entero entre 0 y 4095 (12 bits por defecto)

Ejemplo:

```
const int BUTTON = 4;
const int LED = 2;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON) == LOW) {
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
}
```

⚡ Control PWM con LEDC

La gestión del PWM con LEDC simplifica la configuración: la frecuencia y la resolución quedan ligadas directamente al pin sin gestionar canales explícitamente.

```
// Configura y activa PWM en un pin
ledcAttach(pin, frequency, resolution_bits);

// Establece el ciclo de trabajo
ledcWrite(pin, duty_value);
```

Función	Parámetros	Ejemplo
<code>ledcAttach(pin, freq, res)</code>	Asigna PWM al pin directamente	<code>ledcAttach(5, 8000, 12)</code> → 8 kHz, 12 bits (0-4095)
<code>ledcWrite(pin, duty)</code>	Establece el ciclo de trabajo	<code>ledcWrite(5, 2048)</code> → 50% de ciclo de trabajo

Resolución: 12 bits por defecto (0-4095)

Atenuación LED:

```
const int LED_PIN = 25;

const int frequency = 3000; // 3 kHz

const int resolution = 8; // 8-bit resolution

void setup()
{
    ledcAttach(LED_PIN, frequency, resolution);
}

void loop()
{
    for (int dutyCycle = 0; dutyCycle <= pow(2, resolution) - 1; dutyCycle++)
    {
        ledcWrite(LED_PIN, dutyCycle);
        delay(10);
    }

    for (int dutyCycle = pow(2, resolution) - 1; dutyCycle >= 0; dutyCycle--)
    {
        ledcWrite(LED_PIN, dutyCycle);
        delay(10);
    }

    delay(1000); // Wait for 1 second before starting the next cycle
}
```

2 Interrupciones y temporizadores hardware

 Interrupciones externas (GPIO)

Elemento	Descripción
<code>IRAM_ATTR</code>	Atributo obligatorio en funciones ISR. Guarda la función en RAM interna rápida (IRAM)
<code>attachInterrupt(...)</code>	<code>attachInterrupt(digitalPinToInterrupt(pin), ISR_callback, mode)</code>
Modos disponibles	<code>RISING</code> (flanco ascendente ↑) • <code>FALLING</code> (flanco descendente ↓) • <code>CHANGE</code> (cualquier cambio)

Ejemplo con botón y antirrebote:

```

const int BUTTON_PIN = 4;

volatile int counter = 0;

void IRAM_ATTR handleButtonPress()
{
    counter++;
}

void setup()
{
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    // Configure interrupt on falling edge
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButtonPress,
FALLING);
}

void loop()
{
    Serial.print("Button presses: ");
    Serial.println(counter);
    delay(500);
}

```

⚠ Importante en la ISR:

- Mantén el código rápido y sencillo
- No uses `Serial.print()` (es lento)
- Usa variables `volatile` para datos compartidos

🕒 Temporizadores hardware

Los temporizadores hardware del ESP32 usan una API orientada a la frecuencia y al tiempo en microsegundos.

Configuración típica:

```

hw_timer_t *timer = NULL;

// 1. Inicializar a una frecuencia de conteo en Hz
timer = timerBegin(timer_frequency);

// 2. Vincular la función ISR
timerAttachInterrupt(timer, &timerInterruptISR);

// 3. Configurar la alarma (microsegundos)
timerAlarm(timer, period_us, autoreload, reload_count);

// 4. Control de ejecución

```

```
timerStart(timer);
timerStop(timer);
```

Función	Descripción
<code>timerBegin(freq_hz)</code>	Crea un temporizador a una frecuencia en Hz. Ej: <code>1000000</code> Hz = 1 tick/μs
<code>timerAttachInterrupt(timer, &ISR)</code>	Vincula la ISR para ejecutarse en cada alarma
<code>timerAlarm(timer, period, reload, count)</code>	Configura la alarma: <code>period</code> (μs), <code>reload</code> (bool periódico), <code>count</code> (0=indefinido)
<code>timerStart(timer)</code>	Inicia la cuenta del temporizador
<code>timerStop(timer)</code>	Detiene la cuenta del temporizador

Ejemplo completo - Temporizador periódico:

```
hw_timer_t *timer = NULL;
volatile int counter = 0;

void IRAM_ATTR onTimerAlarm() {
    counter++;
}

void setup() {
    Serial.begin(115200);

    // Temporizador a 1 MHz (1 tick = 1 microsegundo)
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimerAlarm);

    // Alarma cada 1.000.000 μs (1 segundo)
    timerAlarm(timer, 1000000, true, 0);
    timerStart(timer);
}

void loop() {
    if (counter > 0) {
        Serial.printf("Contador: %d\n", counter);
        counter = 0;
        delay(100);
    }
}
```

⚠ Advertencia: límites reales del temporizador ESP32

El temporizador hardware del ESP32 no acepta cualquier frecuencia arbitraria. En la implementación de Arduino ESP32, el divisor utilizado por el temporizador debe mantenerse entre `2` y `65536`, y la fuente de reloj suele ser de `80 MHz` en el ESP32.

Por tanto, el rango real de `timerBegin(freq_hz)` es aproximadamente:

- Frecuencia mínima práctica: $80,000,000 / 65536 \approx 1220,7$ Hz
- Frecuencia máxima práctica: $80,000,000 / 2 = 40,000,000$ Hz = 40 MHz

Esto significa que una frecuencia como 500 Hz está por debajo del rango mínimo soportado para una configuración directa del temporizador. En otras palabras, `timerBegin(500)` no es una configuración válida directamente en el temporizador hardware del ESP32.

Un enfoque válido es configurar el temporizador a una frecuencia base más alta y luego generar el periodo deseado por software. Por ejemplo:

```
// Usa una resolución de 1 MHz para el temporizador
timer = timerBegin(1000000);

// 500 Hz = 2 ms = 2000 us
// Por tanto, la alarma debe durar 2000 microsegundos
timerAlarm(timer, 2000, true, 0);
```

Así, para un objetivo de 500 Hz, la configuración correcta suele ser `timerBegin(1000000)` y `timerAlarm(timer, 2000, true, 0)`, y no `timerBegin(500)`.

Para `timerAlarm(...)`, el periodo se expresa en microsegundos, así que se aplica la misma regla: el temporizador hardware se configura con una frecuencia base soportada y el periodo deseado se obtiene eligiendo el valor adecuado de alarma.

3 Temporizadores software (`Ticker.h`)

Ejecuta tareas periódicas de forma asíncrona mediante temporización software.

Función	Descripción	Ejemplo
<code>Ticker ticker;</code>	Declara un objeto <code>Ticker</code>	-
<code>attach(seconds, callback)</code>	Ejecuta la callback periódicamente (acepta decimales)	<code>ticker.attach(0.5, myFunction)</code>
<code>attach_ms(milliseconds, callback)</code>	Equivalente en milisegundos	<code>ticker.attach_ms(500, myFunction)</code>
<code>detach()</code>	Detiene la ejecución periódica	<code>ticker.detach()</code>

Ejemplo de uso:

```
#include <Ticker.h>

Ticker blinker;
```

```

void blinkLED() {
    digitalWrite(LED_PIN, !digitalRead(LED_PIN));
}

void setup() {
    pinMode(LED_PIN, OUTPUT);
    blinker.attach(0.5, blinkLED);
}

void loop() {
    // Ticker se ejecuta en segundo plano
}

```

✓ Lista de comprobación de ISR (Interrupt Service Routines)

✓ HACER:

- Operaciones rápidas y sencillas
- Usar `IRAM_ATTR` de forma obligatoria
- Usar variables `volatile`
- Usar `xSemaphoreGiveFromISR()` para señalar

✗ NO HACER:

- Operaciones lentas (Serial, WiFi)
- Llamadas a `delay()`
- Asignación de memoria
- `printf()` o funciones pesadas

4 Multitarea y FreeRTOS

El ESP32 integra **dos núcleos de CPU** (Core 0 y Core 1). FreeRTOS permite gestionar hilos concurrentes en ambos.

Creación de tareas y anclaje a un núcleo

```

xTaskCreatePinnedToCore(
    TaskFunction,    // void task(void *param)
    "TaskName",      // Etiqueta de depuración
    StackSize,       // Memoria RAM (palabras de 4 bytes)
    Parameters,       // Parámetros de entrada (normalmente NULL)
    Priority,         // Prioridad (0 = menor)
    &TaskHandle,      // Handle de la tarea
    CoreID            // Núcleo (0 o 1)
);

```

Parámetro	Descripción
TaskFunction	Función <code>void taskName(void *pvParameters)</code>
StackSize	Memoria en bytes (ej: <code>2048</code> = 2 KB)
Priority	0-24 (cuanto mayor, mayor prioridad)
CoreID	<code>0</code> = Core 0 • <code>1</code> = Core 1

Funciones útiles de FreeRTOS

```
// Retardo preciso en tareas periódicas (sin deriva temporal)
vTaskDelayUntil(&xLastWakeTime, xPeriod);

// Obtiene el tick actual del sistema
xLastWakeTime = xTaskGetTickCount();

// Convierte milisegundos a ticks de FreeRTOS
TickType_t xPeriod = pdMS_TO_TICKS(333);

// Devuelve el núcleo en ejecución (0 o 1)
int coreID = xPortGetCoreID();
```

Función	Finalidad	Nota
<code>vTaskDelayUntil()</code>	Retardo periódico preciso	Preferible a <code>delay()</code> en tareas
<code>xTaskGetTickCount()</code>	Obtiene el tick actual	Para sincronización
<code>pdMS_TO_TICKS(ms)</code>	Convierte ms a ticks	Reutilizable
<code>xPortGetCoreID()</code>	Núcleo actual	Útil para depuración

Gestión del ciclo de vida de tareas

Función	Descripción
<code>vTaskDelete(NULL)</code>	Elimina la tarea actual. <code>NULL</code> = propia tarea
<code>vTaskDelete(xTaskHandle)</code>	Elimina una tarea concreta

Tareas de ejecución única:

```
void vTaskOneShot(void *pvParameters) {
    // Realizar trabajo
    Serial.println("Tarea ejecutada una vez");

    // Eliminar tarea para liberar la pila
    vTaskDelete(NULL);
}
```

```
void setup() {
    xTaskCreatePinnedToCore(vTaskOneShot, "OneShot", 2048, NULL, 1, NULL, 0);
}

void loop() {
    vTaskDelete(NULL); // Eliminar loop() en FreeRTOS
}
```

Sincronización y mutex (protección de recursos compartidos)

Conceptos básicos

Sección crítica: Bloque de código donde una tarea accede a un recurso compartido.

Mutex: Mecanismo que permite que solo UNA tarea acceda a un recurso a la vez.

Condición de carrera: Problema que ocurre cuando dos tareas acceden simultáneamente al mismo recurso sin sincronización.

Funciones del mutex

Función	Descripción
<code>xSemaphoreCreateMutex()</code>	Crea un mutex. Devuelve <code>SemaphoreHandle_t</code> o <code>NULL</code> si falla
<code>xSemaphoreTake(mutex, timeout)</code>	Adquiere el mutex (espera si está ocupado). Timeout: <code>portMAX_DELAY</code> = espera infinita
<code>xSemaphoreGive(mutex)</code>	Libera el mutex para que lo usen otras tareas

Valores de retorno:

- `pdTRUE` (1): Operación correcta
- `pdFALSE` (0): Se alcanzó el timeout sin adquirir el mutex

Patrón seguro - con mutex

```
#include <Arduino.h>

SemaphoreHandle_t xMutexSerial;

void vTask1(void *pvParameters) {
    for (;;) {
        // Intentar tomar el mutex
        if (xSemaphoreTake(xMutexSerial, portMAX_DELAY) == pdTRUE) {

            // --- SECCIÓN CRÍTICA PROTEGIDA ---
            Serial.println("[TAREA 1] Usando Serial...");
            vTaskDelay(pdMS_TO_TICKS(100));
            Serial.println("[TAREA 1] Finalizado");
        }
    }
}
```

```

// -----

// Liberar mutex
xSemaphoreGive(xMutexSerial);
}

vTaskDelay(pdMS_TO_TICKS(1000));
}
}

void vTask2(void *pvParameters) {
    for (;;) {
        if (xSemaphoreTake(xMutexSerial, portMAX_DELAY) == pdTRUE) {

            // --- SECCIÓN CRÍTICA PROTEGIDA ---
            Serial.println(" [TAREA 2] Escribiendo...");
            vTaskDelay(pdMS_TO_TICKS(50));
            Serial.println(" [TAREA 2] Hecho");
            // -----

            xSemaphoreGive(xMutexSerial);
        }

        vTaskDelay(pdMS_TO_TICKS(700));
    }
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    // Crear mutex ANTES de las tareas
    xMutexSerial = xSemaphoreCreateMutex();

    if (xMutexSerial != NULL) {
        xTaskCreatePinnedToCore(vTask1, "Task_1", 2048, NULL, 1, NULL, 0);
        xTaskCreatePinnedToCore(vTask2, "Task_2", 2048, NULL, 1, NULL, 1);
    }
}

void loop() {
    vTaskDelete(NULL);
}

```

Ventajas:

-  Escrituras en `Serial` completas y no mezcladas
-  Acceso ordenado a recursos compartidos
-  Evita la corrupción de datos

5 Conectividad (WiFi y MQTT)

Gestión de WiFi (**WiFi.h**)

Función	Descripción	Devuelve
WiFi.begin(SSID, PASSWORD)	Inicializa la conexión al punto de acceso	-
WiFi.status()	Estado del enlace de red	WL_CONNECTED , WL_DISCONNECTED , etc.
WiFi.localIP()	Dirección IP local asignada	Objeto IPAddress
WiFi.disconnect()	Desconecta de la red WiFi	-
WiFi.RSSI()	Intensidad de la señal (dBm)	Negativa, cuanto más cerca de 0 mejor

Ejemplo con reconexión automática:

```
const char *ssid = "MySSID";
const char *password = "MyPassword";
const int TIMEOUT = 10000;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < TIMEOUT) {
    delay(500);
    Serial.print(".");
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nIP: %s | RSSI: %d dBm\n",
      WiFi.localIP().toString().c_str(), WiFi.RSSI());
  } else {
    Serial.println("\nConexión fallida");
  }
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Reconectando WiFi...");
    WiFi.reconnect();
    delay(5000);
  }
}
```

Protocolo MQTT (**PubSubClient.h**)

Comunicación cliente/broker bajo el patrón **Publish/Subscribe**.

Inicialización y conexión:

```
WiFiClient espClient;
PubSubClient client(espClient);

// Asignar broker y puerto
client.setServer("broker.example.com", 1883);

// Registrar callback para mensajes recibidos
client.setCallback(OnMqttReceived);

// Conectar con autenticación (opcional)
client.connect("clientId", "user", "password");

// Bucle de mantenimiento (llamado en setup() o loop())
client.loop();
```

Publicación y suscripción:

```
// Publicar mensaje
client.publish("topic/name", "payload_text");

// Suscribirse al tema
client.subscribe("topic/name");

// Callback para mensajes recibidos
void OnMqttReceived(char *topic, byte *payload, unsigned int length) {
    char message[length + 1];
    memcpy(message, payload, length);
    message[length] = '\0';

    Serial.printf("Tema: %s | Carga útil: %s\n", topic, message);
}
```

Función	Descripción
<code>setServer(host, port)</code>	Define el broker y el puerto MQTT
<code>setCallback(function)</code>	Registra la función para mensajes recibidos
<code>connect(id, [user], [pass])</code>	Conecta y autentica con el broker
<code>loop()</code>	Mantiene la conexión y procesa paquetes
<code>publish(topic, payload)</code>	Publica un mensaje en un tema
<code>subscribe(topic)</code>	Se suscribe a un tema
<code>connected()</code>	Comprueba si está conectado

Función	Descripción
<code>disconnect()</code>	Desconecta del broker MQTT

Patrón con reconexión automática:

```
#include <PubSubClient.h>
#include <WiFi.h>

WiFiClient espClient;
PubSubClient client(espClient);

void reconnectMQTT() {
    if (client.connected()) return;

    Serial.print("Conectando MQTT...");
    if (client.connect("ESP32", "user", "password")) {
        Serial.println(" OK");
        client.subscribe("sensor/temperature");
    } else {
        Serial.printf(" Error: %d\n", client.state());
        delay(5000);
    }
}

void loop() {
    if (!client.connected()) {
        reconnectMQTT();
    }
    client.loop();

    static unsigned long lastTime = 0;
    if (millis() - lastTime > 10000) {
        float temp = 25.5;
        client.publish("sensor/temperature", String(temp).c_str());
        lastTime = millis();
    }
}
```

6 M5Stack Core2: Hardware, pantalla e IMU

La librería `#include<M5Unified.h>` abstrae el hardware integrado de la placa de desarrollo M5Core2.

 Inicialización y gestión de energía

Función	Descripción
<code>M5.begin()</code>	Inicializa el bus I ² C, la pantalla, el chip AXP192 y los periféricos

Función	Descripción
<code>M5.update()</code>	Actualiza el estado de los botones y los eventos táctiles (llamar en <code>loop()</code>)
<code>M5.Axp.SetVibration(bool)</code>	Activa (<code>true</code>) o desactiva (<code>false</code>) el motor de vibración

Pantalla TFT LCD

```
// Pintar fondo
M5.Lcd.fillScreen(BLACK);

// Establecer color de texto y fondo
M5.Lcd.setTextColor(WHITE, BLACK);

// Escala de texto (enteros: 1, 2, 3, etc.)
M5.Lcd.setTextSize(2);

// Posicionar cursor en píxeles (x, y)
M5.Lcd.setCursor(10, 20);

// Escribir texto formateado
M5.Lcd.printf("Valor: %d\n", 42);
M5.Lcd.println("Hola Mundo");
```

Función	Descripción
<code>fillScreen(color)</code>	Pinta la pantalla con el color indicado
<code>setTextColor(text, bg)</code>	Establece el color de texto y fondo
<code>setTextSize(size)</code>	Escala del texto
<code>setCursor(x, y)</code>	Posición en píxeles (origen arriba-izquierda)
<code>printf() / println()</code>	Escribe texto formateado

Colores disponibles: BLACK, WHITE, RED, GREEN, BLUE, YELLOW, CYAN, MAGENTA

Pantalla táctil capacitiva

```
TouchPoint_t pixel_pos;

if (M5.Touch.ispressed()) {
    pixel_pos = M5.Touch.getPressPoint();
    int16_t x = pixel_pos.x; // Coordenada X (0-319)
    int16_t y = pixel_pos.y; // Coordenada Y (0-239)

    Serial.printf("Tacto: (%d, %d)\n", x, y);
}
```

Función	Descripción
<code>M5.Touch.ispressed()</code>	¿Se está tocando la pantalla?
<code>M5.Touch.getPressPoint()</code>	Obtiene las coordenadas del toque (x, y)

Sensor inercial (IMU de 6 ejes)

Acelerómetro y giroscopio integrados en la placa.

```
// Inicializar IMU
M5.IMU.Init();

// Leer aceleraciones (en G)
float accX, accY, accZ;
M5.IMU.getAccelData(&accX, &accY, &accZ);

// Leer orientación estimada (en grados)
float pitch, roll, yaw;
M5.IMU.getAhrsData(&pitch, &roll, &yaw);

Serial.printf("Acc: (%.2f, %.2f, %.2f) G\n", accX, accY, accZ);
Serial.printf("Ori: P=%.1f° R=%.1f° Y=%.1f°\n", pitch, roll, yaw);
```

Función	Parámetros	Unidades
<code>M5.IMU.Init()</code>	-	Inicialización
<code>getAccelData(&x, &y, &z)</code>	punteros float	Aceleración en G
<code>getAhrsData(&p, &r, &y)</code>	punteros float	Ángulos en grados
<code>getGyroData(&x, &y, &z)</code>	punteros float	Velocidad angular en °/s

Ejes de referencia: X (lateral), Y (frontal), Z (vertical)

Funciones útiles

Mapeo y limitación de valores

Función	Descripción	Ejemplo
<code>map(value, fromLow, fromHigh, toLow, toHigh)</code>	Mapea un valor de un rango a otro	<code>map(500, 0, 4095, 0, 320)</code>
<code>constrain(value, min, max)</code>	Limita el valor entre unos límites	<code>constrain(x, 0, 319)</code>
<code>abs(value)</code>	Valor absoluto	<code>abs(-50) → 50</code>
<code>min(a, b)</code>	Mínimo de dos valores	<code>min(10, 20) → 10</code>

Función	Descripción	Ejemplo
<code>max(a, b)</code>	Máximo de dos valores	<code>max(10, 20) → 20</code>

Consejos, buenas prácticas y patrones

Sincronización y mutex

- **CREA siempre el mutex ANTES de las tareas** que lo van a usar
- **Secciones críticas cortas:** Minimiza el tiempo entre `xSemaphoreTake()` y `xSemaphoreGive()`
- Usa `portMAX_DELAY` cuando necesites espera infinita en un mutex crítico
- **Evita anidar mutex:** No tomes el mutex A mientras mantienes el B (deadlock)

Optimización y FreeRTOS

- **Prioridad 0-24:** Cuanto mayor es el número, mayor prioridad. Las tareas de prioridad alta sustituyen a las más bajas
- **Tamaño de pila:** Mínimo 2048 bytes para tareas normales. Aumenta si usas buffers o recursividad
- `vTaskDelete(NULL)`: Llámalo en `loop()` en ESP32/FreeRTOS puro
- `pdMS_TO_TICKS()`: Úsalo siempre para convertir ms a ticks (más portable que codificar valores fijos)
- `vTaskDelayUntil()`: Más preciso que `vTaskDelay()` para tareas periódicas exigentes

Interrupción frente a FreeRTOS

- **ISR (`IRAM_ATTR`):** Para eventos críticos y rápidos (GPIO, temporizadores)
- **Tareas FreeRTOS:** Para tareas largas o control de recursos

Depuración serie

- **115200 baudios:** Estándar. Cambia solo si hay problemas concretos
- `Serial.printf()`: Más flexible que varios `Serial.print()`. Usa especificadores: `%d`, `%f`, `%s`
- **Mutex en Serial:** Obligatorio en multitarea para mensajes completos
- **Timeout en Serial:** `while (!Serial);` en `setup()` para esperar la conexión USB durante el desarrollo

Patrones recomendados

Para aplicaciones críticas:

```
// 1. Crear mutex al inicio
xMutex = xSemaphoreCreateMutex();

// 2. Usarlo en secciones críticas
if (xSemaphoreTake(xMutex, pdMS_TO_TICKS(1000)) == pdTRUE) {
    // Código protegido
    xSemaphoreGive(xMutex);
} else {
    // Timeout: recurso ocupado
    Serial.println("Timeout!");
}
```

Documentación y recursos

Referencias oficiales

- [Documentación oficial de Arduino ESP32](#)
- [Manual de referencia de FreeRTOS](#)
- [Documentación de M5Stack](#)
- [Manual de referencia técnica de ESP32](#)

Temas clave por sección

Sección	Concepto principal	Caso de uso
GPIO digital/analógica	Entrada/salida de bits y valores analógicos	LEDs, botones, sensores
Interrupciones	Respuesta rápida a eventos hardware	Detección rápida de cambios
Temporizadores	Generación de eventos periódicos hardware	PWM, muestreo periódico
Ticker	Tareas periódicas software	Actualización no crítica de estado
Multitarea FreeRTOS	Ejecución concurrente controlada	Múltiples procesos simultáneos
Mutex/Sincronización	Protección de recursos compartidos	Acceso seguro a Serial, LCD, GPIO
WiFi/MQTT	Conectividad de red	Comunicación remota, IoT
M5Core2	Plataforma de desarrollo integrada	Aplicaciones con pantalla y sensores

Ruta recomendada de aprendizaje

1. **Nivel básico:** GPIO → Serial → Interrupciones
2. **Nivel intermedio:** Temporizadores → Ticker → FreeRTOS básico
3. **Nivel avanzado:** Mutex → Multitarea coordinada → WiFi/MQTT
4. **Aplicaciones:** M5Core2 → Integración de proyectos

Herramientas útiles

- **Arduino IDE:** Descárgalo en arduino.cc
- **PlatformIO:** Alternativa potente en VS Code
- **Mosquitto:** Broker MQTT local para pruebas
- **Wokwi:** Simulador online de ESP32 (wokwi.com)

Consejo de aprendizaje

- **Lee el código antes de ejecutarlo:** Comprende qué hace cada función
- **Experimenta con los valores:** Cambia tiempos, prioridades y periodos
- **Monitoriza con Serial:** Usa `Serial.printf()` para depurar continuamente
- **Aísla los problemas:** Prueba cada componente por separado antes de integrarlos
- **Documenta tu trabajo:** Comenta el código y entiende por qué funciona

Última actualización: 2026-07-28 | **Versión:** 2.0 | **Para:** Grado en Ingeniería Electrónica Industrial

- [Hoja de datos del ESP32](#)